

INFORME TÉCNICO

Aplicación Android — Pico Botella (PB)

Repositorio: <https://github.com/StiviMoon/PB>

Plataforma: Android (Kotlin) | Arquitectura: MVVM | Versión: 1.1.0

1. Descripción General del Proyecto

"Pico Botella" es una aplicación de entretenimiento para Android que simula el clásico juego de la botella. Los jugadores se ubican alrededor del dispositivo, presionan el botón parpadeante para girar la botella virtual y el jugador señalado debe cumplir el reto que la aplicación muestre de forma aleatoria. La app permite gestionar una lista personalizada de retos guardados localmente en SQLite mediante Room, y consume una API externa de Pokémon para hacer la experiencia visual más entretenida.

2. Tecnologías y Dependencias Utilizadas

Tecnología	Propósito
Kotlin	Lenguaje principal de desarrollo
Android SDK 36 (mín. 24)	SDK objetivo — compatible con Android 7.0+
MVVM + StateFlow	Patrón de arquitectura: separa lógica de UI
Room (SQLite)	Base de datos local para persistir los retos
Kotlin Coroutines	Operaciones asíncronas sin bloquear la interfaz
Navigation Component	Navegación entre Fragments
ViewBinding	Acceso seguro a vistas XML desde Kotlin
ViewModel + LiveData	Estado persistente ante rotaciones de pantalla
MediaPlayer	Reproducción de música de fondo y sonido de giro
HttpURLConnection	Petición HTTP a la API de Pokémon
Material3 (Material Design)	Componentes visuales modernos de Google

3. Estructura del Proyecto

PB-Github/

■■■ app/src/main/

■■■ java/com/example/pb/

■ ■■■ MainActivity.kt

← Actividad única (Single Activity)

■ ■■■ data/

■ ■■■ AppDatabase.kt

← Configuración de Room (Singleton)

■ ■■■ RetoDao.kt

← Consultas SQL para la tabla retos

■ ■■■ model/

■ ■■■ Reto.kt

← Entidad de base de datos

■ ■■■ repository/

```

■ ■ ■ ■ ■ RetoRepository.kt      ← Intermediario DAO ↔ ViewModel
■ ■ ■ ■ ■ PokemonRepository.kt  ← Consumo de API externa de Pokémon
■ ■ ■ ■ ■ ui/
■ ■ ■ ■ ■ splash/SplashFragment.kt
■ ■ ■ ■ ■ home/HomeFragment.kt
■ ■ ■ ■ ■ instrucciones/InstruccionesFragment.kt
■ ■ ■ ■ ■ retos/
■ ■ ■ ■ ■ RetosFragment.kt
■ ■ ■ ■ ■ RetosAdapter.kt
■ ■ ■ ■ ■ AgregarRetoDialog.kt
■ ■ ■ ■ ■ EditarRetoDialog.kt
■ ■ ■ ■ ■ toolbar/CustomToolbarView.kt
■ ■ ■ ■ ■ utils/AnimationUtils.kt
■ ■ ■ ■ ■ viewmodel/
■ ■ ■ ■ ■ RetosViewModel.kt
■ ■ ■ ■ ■ AudioViewModel.kt
■ ■ ■ ■ ■ res/
■ ■ ■ ■ ■ layout/      ← Diseños de pantallas y diálogos
■ ■ ■ ■ ■ drawable/    ← Íconos y fondos personalizados (shapes)
■ ■ ■ ■ ■ raw/         ← Archivos de audio (.ogg)
■ ■ ■ ■ ■ values/      ← Colores, strings, temas

```

4. Arquitectura MVVM

La aplicación implementa el patrón **MVVM (Model-View-ViewModel)**, el estándar recomendado por Google para apps Android modernas. El flujo de datos es el siguiente:

```

[Fragment / Vista] ←-observa StateFlow■ [ViewModel] ←-llama■ [Repository] ←-consulta■ [Room / API]

```

- **Vista (Fragment):** Solo muestra datos y notifica acciones del usuario. No contiene lógica de negocio.
 - **ViewModel:** Contiene la lógica, expone el estado con StateFlow. Sobrevive a rotaciones de pantalla.
 - **Repository:** Fuente única de verdad. Abstrae si los datos vienen de Room o de una API externa.
 - **Room/DAO:** Convierte consultas Kotlin en SQL real y retorna Flow reactivos que actualizan la UI automáticamente.
-

5. Modelo de Datos — Entidad Reto

```

@Entity(tableName = "retos")
data class Reto(
    @PrimaryKey(autoGenerate = true)
    val id: Int = 0,
    val descripcion: String,
    val timestamp: Long = System.currentTimeMillis()
)

```

La tabla **retos** en SQLite tiene 3 columnas: **id** (clave primaria autoincremental), **descripcion** (texto del reto escrito por el usuario), y **timestamp** (fecha de creación en milisegundos, guardada automáticamente). La base de datos está en la versión 2 y usa **fallbackToDestructiveMigration** para recrarse sin errores si el esquema cambia.

6. Base de Datos — RetoDao (Consultas SQL)

Función	Consulta SQL	Descripción
getAll()	SELECT * FROM retos ORDER BY id DESC	Lista todos los retos, los más nuevos primero. Retorna un Flow reactivo.

Función	Consulta SQL	Descripción
getRandomReto()	SELECT * FROM retos ORDER BY RANDOM()	Retorna un reto aleatorio. Usado para la HU 12.
insert(retos)	INSERT	Guarda un nuevo reto en la tabla.
update(retos)	UPDATE	Modifica la descripción de un reto existente.
delete(retos)	DELETE	Elimina un reto específico de la tabla.

7. Funcionalidades Implementadas (Historias de Usuario)

HU 6.0 — Agregar y Listar Retos

Archivos: RetosFragment.kt, AgregarRetoDialog.kt, RetosAdapter.kt, RetosViewModel.kt

Muestra todos los retos en un RecyclerView ordenados del más nuevo al más antiguo. Un FAB naranja abre el diálogo para agregar un reto. El botón "Guardar" solo se activa cuando el campo de texto no está vacío. Al guardar, la lista se actualiza inmediatamente gracias al Flow reactivo de Room.

HU 8.0 — Editar Reto

Archivos: EditarRetoDialog.kt, dialog_editar_reto.xml, RetosViewModel.kt

Cada ítem tiene un ícono de lápiz. Al pulsarlo, se abre un diálogo con la descripción actual del reto en un campo editable. "Cancelar" cierra sin cambios; "Guardar" actualiza el reto en SQLite. El diálogo no se cierra si se toca fuera (setCanceledOnTouchOutside(false)).

HU 9.0 — Eliminar Reto (cuadro de diálogo de confirmación)

Archivos: RetosFragment.kt, dialog_delete_reto.xml, RetosViewModel.kt

Cada ítem tiene un ícono de tacho. Al pulsarlo, aparece un cuadro de diálogo con fondo blanco, el título "¿Desea eliminar el siguiente reto?" en negro y negrita, y la descripción exacta del reto. Los botones "NO" y "SI" son textos en color naranja (#FF6F00). "NO" cierra el diálogo; "SI" llama a viewModel.eliminarReto(retos) que ejecuta DELETE en SQLite y la lista se actualiza al instante. El diálogo no se cierra al tocar fuera (setCancelable(false)).

HU 10.0 — Compartir la Aplicación

Archivos: HomeFragment.kt, CustomToolbarView.kt

El toolbar tiene un ícono de compartir. Al pulsarlo, se lanza un Intent ACTION_SEND de tipo "text/plain" que abre el selector nativo de Android para compartir el link de la app con un mensaje predefinido.

HU 11.0 — Sonido de Botella Girando

Archivos: AudioViewModel.kt, HomeFragment.kt, res/raw/spin_sound.ogg

Al girar la botella se reproduce un efecto de sonido (spin_sound.ogg). El AudioViewModel gestiona dos instancias de MediaPlayer separadas: una para la música de fondo (bgMediaPlayer) y otra para el sonido de giro (spinMediaPlayer), lo que permite que funcionen de forma independiente.

HU 12.0 — Mostrar Reto Aleatorio

Archivos: HomeFragment.kt, dialog_random_reto.xml, PokemonRepository.kt

Tras la animación de giro aparece un diálogo con fondo oscuro, bordes blancos redondeados y transparencia. En la parte superior sobresale un círculo (borde blanco, fondo negro) con la imagen de un Pokémon aleatorio. En el centro, el texto del reto aleatorio en blanco y negrita (traído con dao.getRandomReto()). Un botón naranja "Cerrar" en la parte

inferior permite volver al juego. El diálogo NO se cierra al tocar fuera (`setCancelable(false)`).

Imagen Pokémon: `PokemonRepository` realiza una petición HTTP GET a la API `https://raw.githubusercontent.com/Biuni/PokemonGO-Pokedex/master/pokedex.json`, parsea el JSON, selecciona un Pokémon al azar y retorna la URL de imagen. La descarga corre en `Dispatchers.IO` (hilo secundario) para no bloquear la UI.

8. Navegación Entre Pantallas

La app es **Single Activity**: una sola `MainActivity` con un `NavHostFragment`. La navegación se gestiona con el **Navigation Component** de Android Jetpack.

Origen	Destino	Disparador
Splash	Home	Automático — delay de 5 segundos con animación de botella
Home	Retos	Botón de retos en el toolbar personalizado
Home	Instrucciones	Botón de instrucciones en el toolbar
Retos	Home	Flecha de volver en la toolbar de Retos

9. Sistema de Audio — `AudioViewModel`

Función	Descripción
<code>startBgMusic()</code>	Inicia la música de fondo en bucle (si el audio está activado)
<code>pauseBgMusic()</code>	Pausa la música al salir de la pantalla (<code>onPause</code>)
<code>resumeIfEnabled()</code>	Reanuda la música si el usuario no la silenció
<code>toggleAudio()</code>	Activa/Desactiva el audio. El ícono del toolbar se actualiza automáticamente.
<code>playSpinSound()</code>	Reproduce el sonido de giro de la botella (HU 11.0)
<code>stopSpinSound()</code>	Detiene y libera el <code>MediaPlayer</code> del sonido de giro
<code>onCleared()</code>	Libera ambos <code>MediaPlayers</code> cuando el <code>ViewModel</code> se destruye

10. Toolbar Personalizada — `CustomToolbarView`

Es un componente de vista personalizado que extiende **`LinearLayout`**. Encapsula los 5 botones del toolbar y expone listeners mediante lambdas. Todos los botones tienen una micro-animación de escala (`touch_scale.xml`) al ser presionados.

Botón	Acción
Calificar (estrella)	Abre la app en Play Store mediante un Intent <code>ACTION_VIEW</code>
Audio (parlante)	Llama a <code>audioViewModel.toggleAudio()</code> y actualiza el ícono
Instrucciones (libro)	Navega a <code>InstruccionesFragment</code>
Retos (trofeo)	Navega a <code>RetosFragment</code>
Compartir	Lanza el Intent de compartir (HU 10.0)

11. Paleta de Colores y Tema Visual

El tema base es **Theme.Material3.DayNight.NoActionBar**, que desactiva la barra de acción nativa y le da control total al diseño personalizado.

Nombre	HEX	Uso en la app
brand_primary / orange	#FF6600	Botones principales, acentos, íconos
brand_primary_dark / orange_dark	#CC5200	Estado presionado del naranja
bg_home	#3D1F0A	Fondo de la pantalla de juego (marrón/madera)
bg_screen	#111111	Fondo de pantallas secundarias (gris oscuro)
bg_toolbar / black	#000000	Fondo del toolbar y Splash
text_primary / white	#FFFFFF	Texto principal
text_secondary	#B0B0B0	Texto secundario / mensajes vacíos

12. Resumen de Archivos Clave

Archivo .kt	Responsabilidad
MainActivity.kt	Punto de entrada; solo infla el layout con NavHostFragment
AppDatabase.kt	Singleton de Room; configura la BD 'pb_database' versión 2
RetoDao.kt	CRUD completo + consulta aleatoria sobre la tabla retos
Reto.kt	Modelo de datos: id, descripcion, timestamp
RetoRepository.kt	Intermediario DAO ↔ ViewModel para los retos
PokemonRepository.kt	Consume la API de Pokémon y retorna URL de imagen aleatoria
RetosViewModel.kt	Lógica: insertar, editar, eliminar retos; estado como StateFlow
AudioViewModel.kt	Gestión completa de audio: fondo y sonido de giro
HomeFragment.kt	Pantalla principal; animación de botella; dispara el reto aleatorio
RetosFragment.kt	Lista retos; maneja el diálogo de eliminación (HU 9.0)
AgregarRetoDialog.kt	Diálogo para crear un nuevo reto (HU 6.0)
EditarRetoDialog.kt	Diálogo para editar un reto existente (HU 8.0)
RetosAdapter.kt	Adaptador del RecyclerView; gestiona clics Editar y Eliminar
CustomToolbarView.kt	Vista personalizada del toolbar con 5 acciones
SplashFragment.kt	Pantalla de carga de 5 segundos con animación de botella